# Lesotho Farmers Portal — Technical Overview

A Joget DX Enterprise implementation of the GovStack Registration Building Block

Prepared for Joget Inc. · 24 August 2026 · FiscalAdmin Ltd

---

## 1. The service

The Farmers Portal is the national farmer and land-parcel registry of the Kingdom of Lesotho, owned and operated by the Ministry of Agriculture, Food Security and Nutrition (MAFSN). It was launched nationally on 24 August 2026 at Moyeni, Quthing, by the Honourable Minister of Agriculture, Food Security and Nutrition.

Before this system there was no single national record of who farms in Lesotho or where. Farmer data existed as paper forms, spreadsheets and district-level fragments. The portal replaces that with one registry, held by the Ministry, keyed on the National ID, and reachable by field officers in all ten districts over the public internet.

At launch the registry supports farmer registration through an eight-page wizard, parcel registration with GPS boundary capture, and registry search and supervisory review.

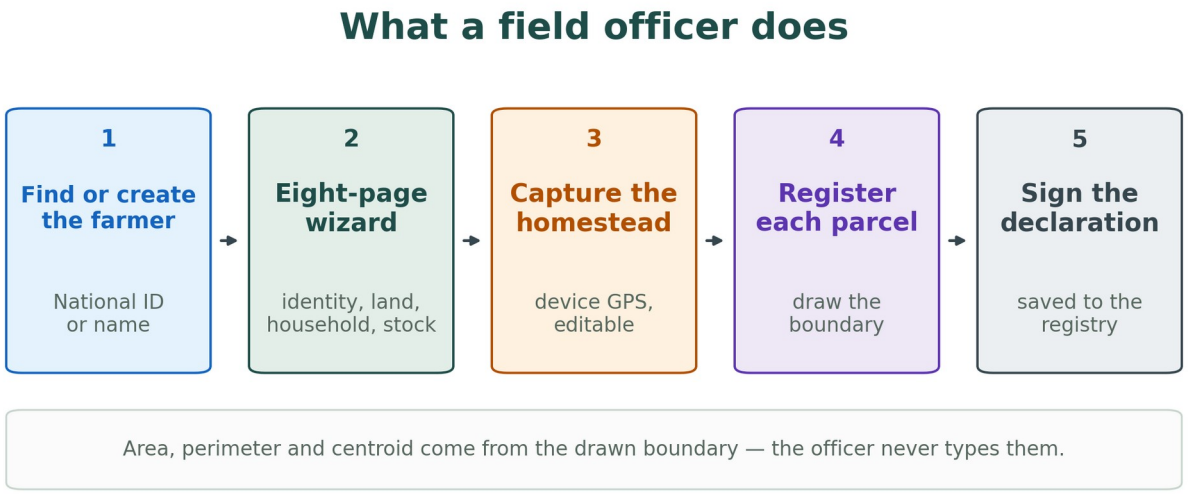



*Figure 1 — the registration journey as the field officer experiences it.*

## 2. Who built it, and with whom

The system was delivered by **FiscalAdmin Ltd**. It exists because several partners each supplied something the others could not.

- **MAFSN** owns the registry, the data and the process, and supplied the domain.
- **The International Telecommunication Union (ITU)** provided the programme frame and the digital-government methodology, and has carried the Lesotho use case into its GovStack sandbox and knowledge-product work.
- **GovStack** supplied the architecture: the Registration Building Block specification and the PAERA reference architecture that sits above it.

- **The World Food Programme (WFP)** supported the implementation, including the Joget DX Enterprise licence under which the system runs.
- **Vodacom Lesotho** partnered on the launch and on reaching officers in the field.

## 3. The GovStack frame — why the system has the shape it has

This is not a bespoke system that was later described in GovStack vocabulary. The relationship runs the other way.

From the GovStack side, the Farmers Portal was set up from the beginning as a **reference implementation of the Registration Building Block** — a deliberate test of whether the specification could carry a real national service in a country with modest ICT capacity, rather than only a demonstrator. It is now among the specification's completed, live implementations, and that is the substantive result: the RegBB is no longer only a document.

Above the building blocks sits **PAERA**, the GovStack Public Administration Ecosystem Reference Architecture (v1.0, 2024) — the framework that explains how a public administration as a whole should be structured for digital transformation, and which the implementation followed. Its lead author is also the architect of this system, which is why the loop below closes rather than merely being asserted.

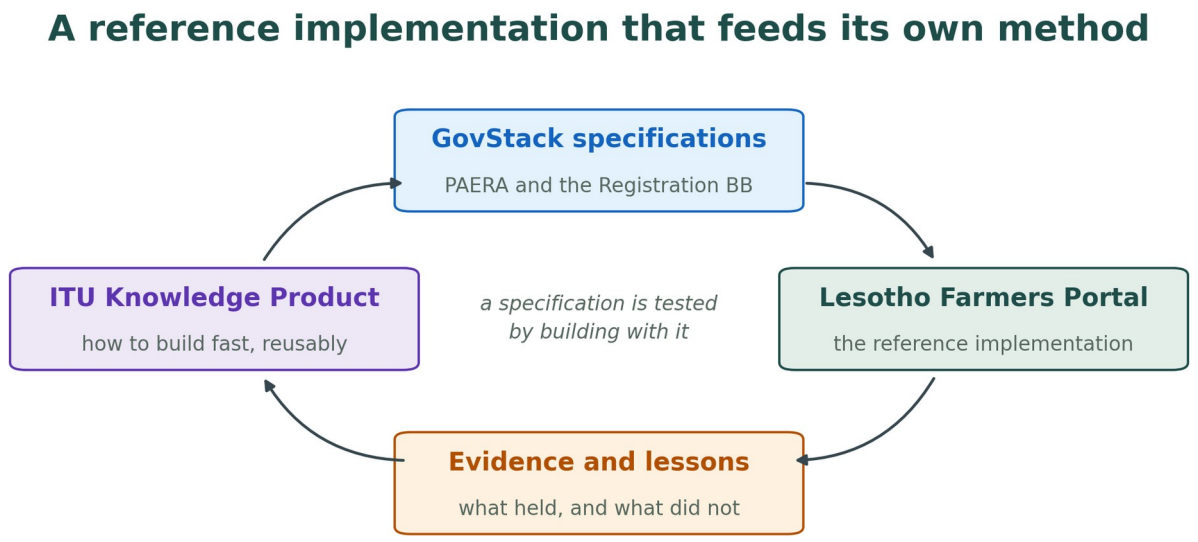



*Figure 2 — the specification, the implementation and the knowledge product feed each other.*

## 4. Scope at launch, and what follows

The platform as built is wider than the service that went live. Programme design, subsidy application, eligibility evaluation, decision management, entitlement issuance, budget control and input distribution by voucher are all implemented and tested, and their tables and screens are present in the deployed application.

The Ministry's decision was to launch registration first. Building the registry is the precondition for everything else — there is no point evaluating a subsidy application against a registry that does not yet hold the farmer. The subsidy lifecycle follows once registration has national coverage.

This document therefore separates what is **live and in daily use** from what is **built and staged**. Both are real; only the first has farmers in it.

## 5. Platform and environment

| | |
|---|---|
| Application platform | Joget DX Enterprise 9.0.4 |
| Servlet container | Apache Tomcat 11 |
| Database | PostgreSQL (Azure-managed) |
| Deployment | Docker on an Azure virtual machine |
| Edge | nginx, TLS via Let's Encrypt, HTTP redirected to HTTPS |
| Public address | the Ministry's national e-services domain |
| Plugin runtime | OSGi, plugins compiled to Java 11 bytecode |
| Source control | Git — the whole application is configuration-as-code |

One note of practical interest to Joget: the custom plugins are built against the Joget 8.1 plugin API and load without modification on DX 9 through Joget's Jakarta EE migration layer. All twelve production bundles report Active with no failed starts. That binary compatibility across a major platform version saved a rebuild cycle at a point in the project where there was no time for one.

## 6. Platform, plugins and configuration

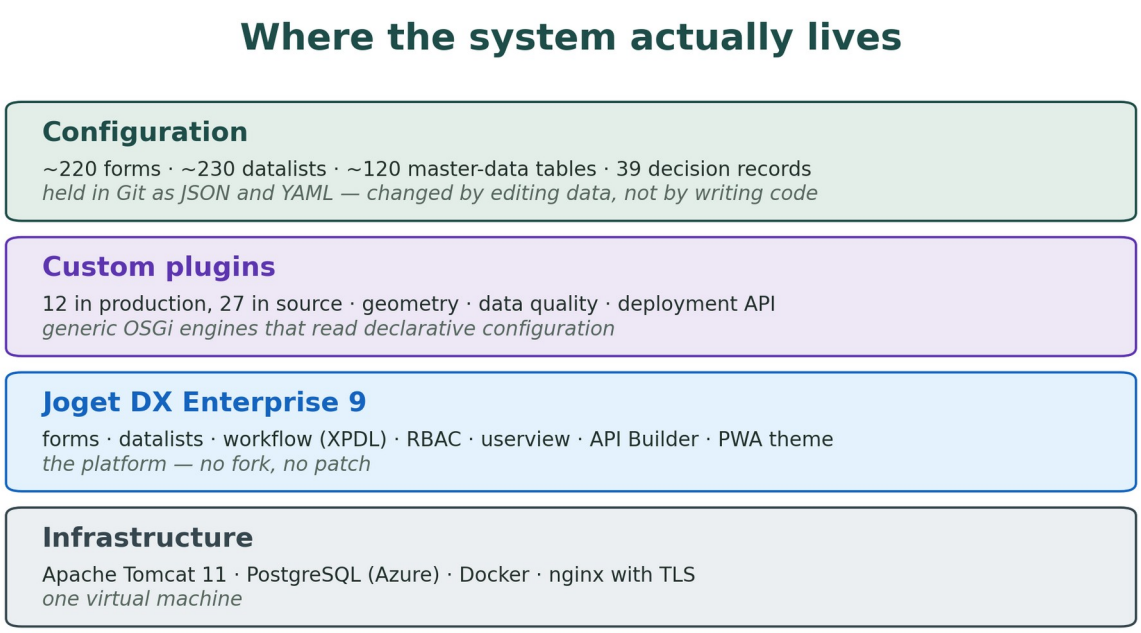



*Figure 3 — four layers, and the effort is concentrated in the top one.*

### What Joget DX carried natively

The great majority of the system is Joget configuration rather than code:

- **Form engine** — approximately 220 form definitions, including multi-page wizards, tab subforms, repeating grids, cascading selects bound to master-data forms, field validators and calculated fields.
- **Datalist engine** — approximately 230 list definitions with filters, inline actions and row-level permissions.

- **Userview** — a single navigation application with role-scoped categories and menus.
- **Workflow engine** — XPDL process definitions for the application lifecycle.
- **RBAC** — Joget users, groups and userview permissions, mapped onto five operational roles: System Administrator, District Supervisor, Field Officer, Finance Officer, Analyst.
- **Master data** — approximately 120 reference tables (districts, community councils, villages, crops, livestock types, agro-ecological zones, document types, statuses, rejection reasons and the rest), each a standard Joget form with a standard CRUD screen, editable by Ministry staff without developer involvement.
- **API Builder** — the credentialed REST surface used by the custom widgets and the deployment tooling.
- **Theme and PWA shell** — the mobile-usable interface field officers work in.

### What we built as custom plugins

Twenty-seven OSGi plugin modules live in the source tree. Twelve are deployed in production; the remainder support the subsidy lifecycle and are staged for the next phase.

| Plugin | What it does |
|---|---|
| `form-creator-api` | REST deployment of form, datalist and userview definitions through Joget's own DAO, so configuration can be version-controlled and pushed rather than hand-edited in the console |
| `reg-bb-engine` | Interprets the service metamodel at runtime — the component that makes new programmes a data change rather than a code change |
| `reg-bb-publisher` | Publishes registry events and the metamodel to consumers |
| `joget-gis-server` | Backend geometry services — polygon validation, overlap detection, area, perimeter and centroid |
| `joget-gis-ui` | Leaflet-based polygon capture element used on the parcel form |
| `parcel-zone-centring` | Auto-centres the map on the farmer's village or resource centre so the officer does not start from a blank world map |
| `joget-smart-search` | Typeahead search element with configurable display columns and filter chains — the "Find Farmer" control |
| `joget-concat-field` | Derives a value from several source fields |
| `embedded-datalist` | Read-only inline list inside a form, filtered by the parent record |
| `joget-advanced-filters` | Cascading master-data and date-range filter types for datalists |
| `form-quality-runtime` | Configurable data-quality rule engine — SQL-defined rules, severity-aware issue register, per-record quality status and status-transition gating, all administered from seven configuration forms |
| `farmer-derived-plugin` | Derived and aggregate values on the farmer record |

**Built and staged for the subsidy phase:** `application-engine-runtime` (dynamic application forms seeded from the chosen programme's spec), `decision-engine-runtime` (operator decisions, entitlement issuance, decision audit), `identity-resolver-runtime` (configurable resolution of a person by foundational identifier, with field projection into the calling form), `joget-status-framework` (generic entity lifecycle and transition audit), the eligibility DSL stack (`rules-grammar`, an ANTLR 4 parser; `joget-rules-api`, which compiles the DSL to SQL; `joget-rule-editor`, the operator-facing authoring UI; `subsidy-eligibility-runtime`, the per-applicant evaluator), and the GovStack submission backbone (`wf-activator`, `doc-submitter`, `processing-server`).

The design rule applied throughout: **a plugin is a generic engine that reads declarative configuration.** None of the platform-tier plugins knows what a farmer is. That is what makes the "configuration over code" claim survive contact with a second programme.

## 7. Configuration as code

The entire application — every form, datalist, userview and master-data seed — is held in Git as JSON and YAML and pushed to an instance through `form-creator-api`. Direct SQL writes to Joget metadata tables are prohibited by project rule, because they bypass Joget's definition cache and its per-form ORM mapping and can silently lose data.

The consequences are worth stating plainly, because they are the reason a one-person team could run a national service:

- A release is a Git commit plus a scripted push. It is reviewable and repeatable.
- Two instances (a local replica and production) can be diffed and reconciled.
- A regression suite of 311 automated tests runs against a live instance in about a minute.
- A rollback is a re-push of the previous definitions, with a database dump as the backstop.

## 8. How it was built — LLM-assisted spec-driven development

The delivery method is itself part of what this project produced, and it is the part most likely to transfer.

The build used **LLM-assisted spec-driven development**. The specification — a YAML form specification, a service contract, a seed file, an architecture decision record — is the primary artefact and the thing under review. An LLM assistant works *against* that specification to author forms, plugin code and seed data; generation is deterministic; the result is pushed through the platform's own API; and an automated regression suite decides whether the change is done. When something fails, the correction goes back into the specification, never into a hand-patched instance. Thirty-nine architecture decision records carry the reasoning that the specifications themselves do not.

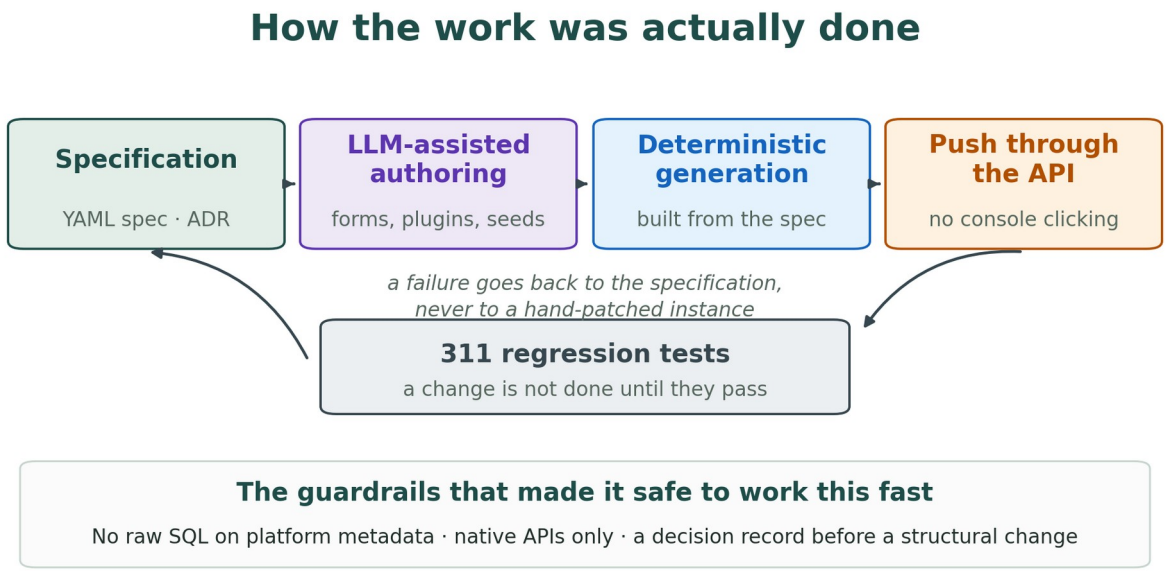



*Figure 4 — specification first, generation deterministic, tests as the gate.*

What made this safe rather than reckless was the guardrails, and they are worth naming because they are the transferable part: no raw SQL against platform metadata; platform-native APIs only; a decision record before any structural change; the repository updated before any instance. An assistant that is fast and occasionally wrong is an asset inside those constraints and a liability outside them.

The practical experience of running this method on a real national service — what accelerated, what did not, where the guardrails earned their cost — is feeding an **ITU Knowledge Product** on how to build public-sector applications quickly and reusably using the PAERA methodology and the GovStack building-block approach. The Farmers Portal is the worked example behind that guidance.

## 9. GovStack RegBB conformance

The registry implements the seven RegBB concerns as distinct, configurable, API-addressable components rather than as hard-coded application logic. An internal conformance review in May 2026 assessed the system as **substantially conformant** at the first horizon, with the remaining gaps deliberately phased: a systematic OpenAPI-published REST surface, and connectors to external GovStack Building Blocks for Identity, Messaging and Payments.

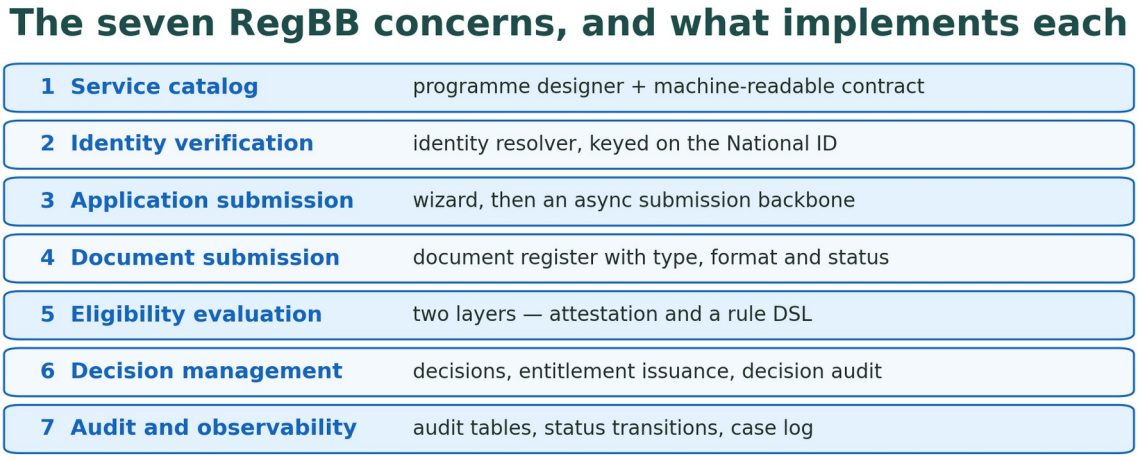



*Figure 5 — the seven concerns and the components that implement them.*

Two elements are worth singling out for a technical audience.

**A machine-readable service contract.** Each service is described by a `{serviceId}.yml` metamodel that declares the entity model, identifier types, master-data references, normalisation rules and a field-by-field translation between Joget field ids and GovStack JSON paths. Sender and receiver load the same file. This is what makes the citizen-facing portal and a back-office system separable rather than notionally decoupled.

**Two-layer eligibility.** Citizen self-attestation at data entry and authoritative server-side evaluation against the registry at submission are treated as different concerns and run at different lifecycle stages.

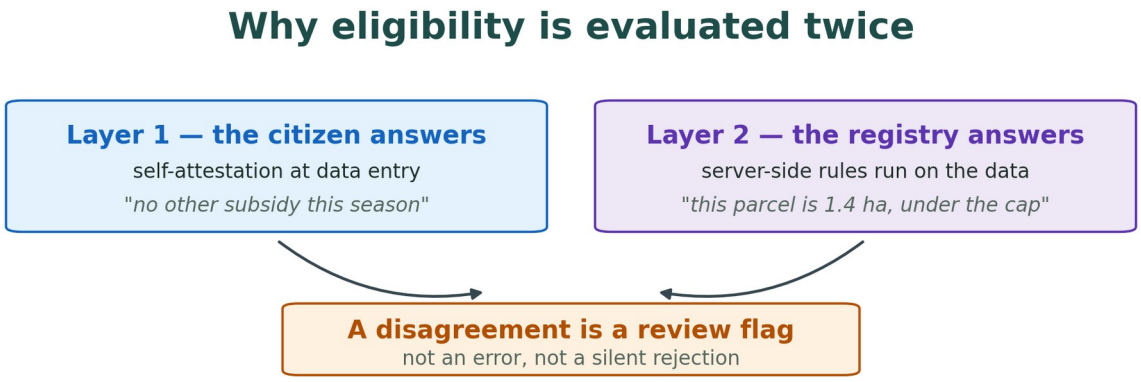



*Figure 6 — the two layers answer different questions, and disagreement is information.*

## 10. Why the low-code platform was decisive

This is the part most likely to be useful to other public-sector programmes, so it is stated without varnish.

The Ministry's ICT department is small. The delivery team was, for most of the project, one architect-developer. The elapsed time from prototype to a production national service was about two years. A conventional build — bespoke framework, hand-written CRUD, a custom admin console for 120 reference tables, a bespoke permissions model — would not have fitted inside either constraint.

What the platform actually bought:

1. **The generic surface came free.** Forms, lists, navigation, RBAC, workflow, audit hooks and a mobile-capable theme were configuration, not code. The effort went into the parts that are genuinely specific to agriculture in Lesotho: geometry capture, data quality, the metamodel.
2. **The extension points were sufficient.** Everything the domain needed that the platform did not provide could be added as an OSGi plugin without forking or patching the platform.
3. **The Ministry can maintain what it now owns.** Master data, reference lists, resource centres, document types and rejection reasons are edited through ordinary screens. A change that would otherwise be a support ticket and a release is a Ministry officer editing a row.

The honest counterpart: the configuration-as-code discipline in section 7 and the guardrails in section 8 were not optional. A national service built by clicking in a console, with no version control and no scripted deployment, would not have been recoverable when something went wrong. The platform made the build possible; the engineering discipline made it operable.

## 11. Facts and figures

| | |
|---|---|
| Launched | 24 August 2026, Moyeni, Quthing |
| Coverage | All 10 districts of Lesotho |
| Standard implemented | GovStack Registration Building Block, under PAERA |
| Form definitions | ~220 |
| Datalist definitions | ~230 |
| Master-data reference tables | ~120 |
| Custom OSGi plugin modules | 27 in source, 12 deployed in production |
| Architecture decision records | 39 |
| Automated regression tests | 311 |
| Operational roles | 5 |
| Delivery team | One architect-developer, with intermittent support |
| Elapsed build time | Approximately two years, prototype to national production |

## 12. Use of this material

The system belongs to the Ministry of Agriculture, Food Security and Nutrition of the Kingdom of Lesotho. This overview is offered freely as source material for a Joget case study, and FiscalAdmin is glad to be named and quoted. Any publication that uses the Ministry's name, screenshots or data should be cleared with MAFSN first; FiscalAdmin will help route a draft to the Ministry's ICT department. Where the GovStack, PAERA or ITU roles are described, please keep the attribution above intact — the partnership is part of what the result demonstrates.

Technical questions are welcome and will be answered directly.

**Aare Lapõnin** · FiscalAdmin Ltd